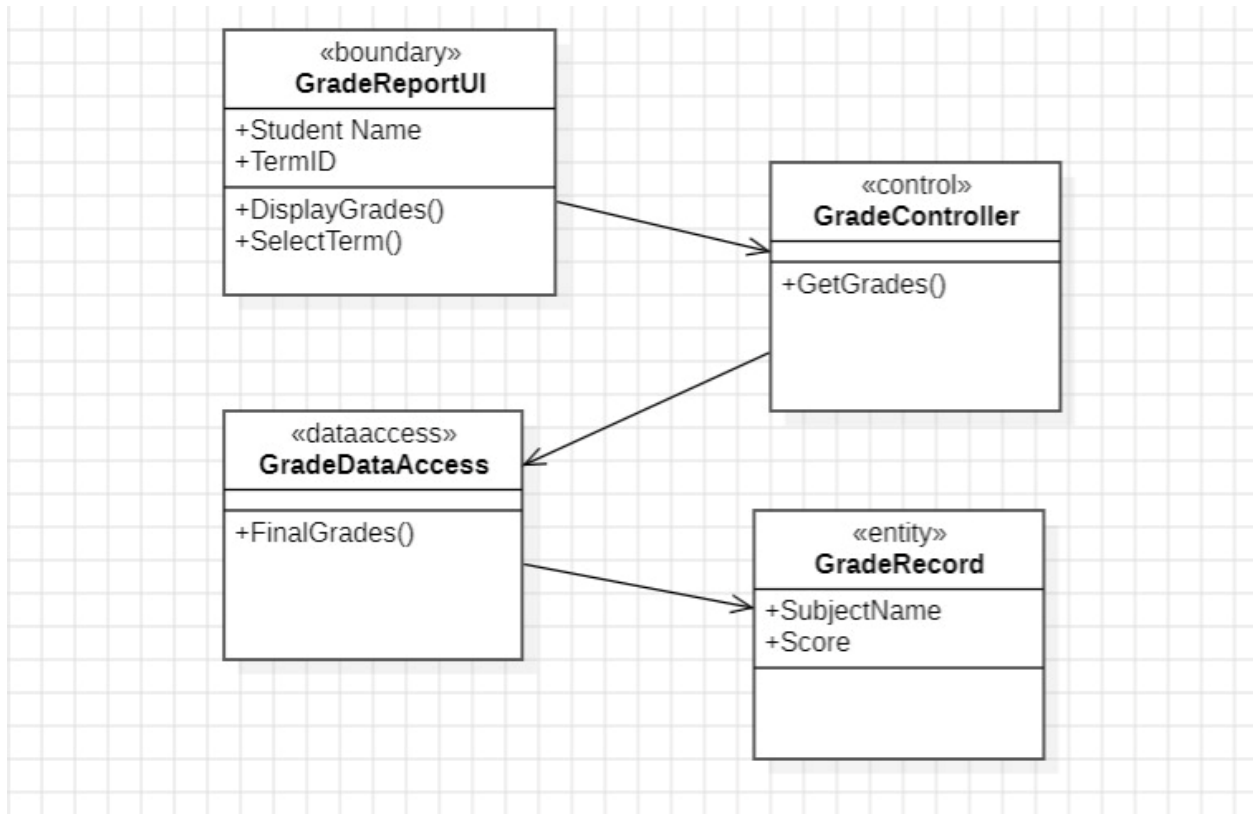


Object-Oriented Design

1. Use Case Name: View Grades (Simple)

- Scenario: Student or Parent checks academic performance for a specific term.
- Triggering event: User selects the “View Grades” option from the dashboard.
- Brief description: Retrieves and displays a student's grades and calculated GPA for a selected academic term.
- Actors: Primary: Student or Parent. Secondary: Registrar (data source).
- Stakeholders: Students; Parents; Teachers; Academic Advisors.
- Related use cases: Enter or Update Grades (grades must be entered first).
- Preconditions: * User is authenticated and logged into the system.
 - Grade records for the requested term have been published.
- Postconditions: * Grade report is displayed; GPA is calculated and shown.
- Flow of activities:
 1. Actor requests “View Grades” and selects a specific academic term.
 2. System (GradeController) validates the student session.
 3. System (GradeDataAccess) queries the database for matching GradeRecords.
 4. System calculates the term GPA based on retrieved scores and credits.
 5. System displays the grade list and GPA summary on the GradeReportUI.
- Exception conditions:
 - No grades found for selected term → display "No records available" message.
 - Session timeout → redirect to Login screen.

First Cut Design Class Diagram :



CRC Cards :

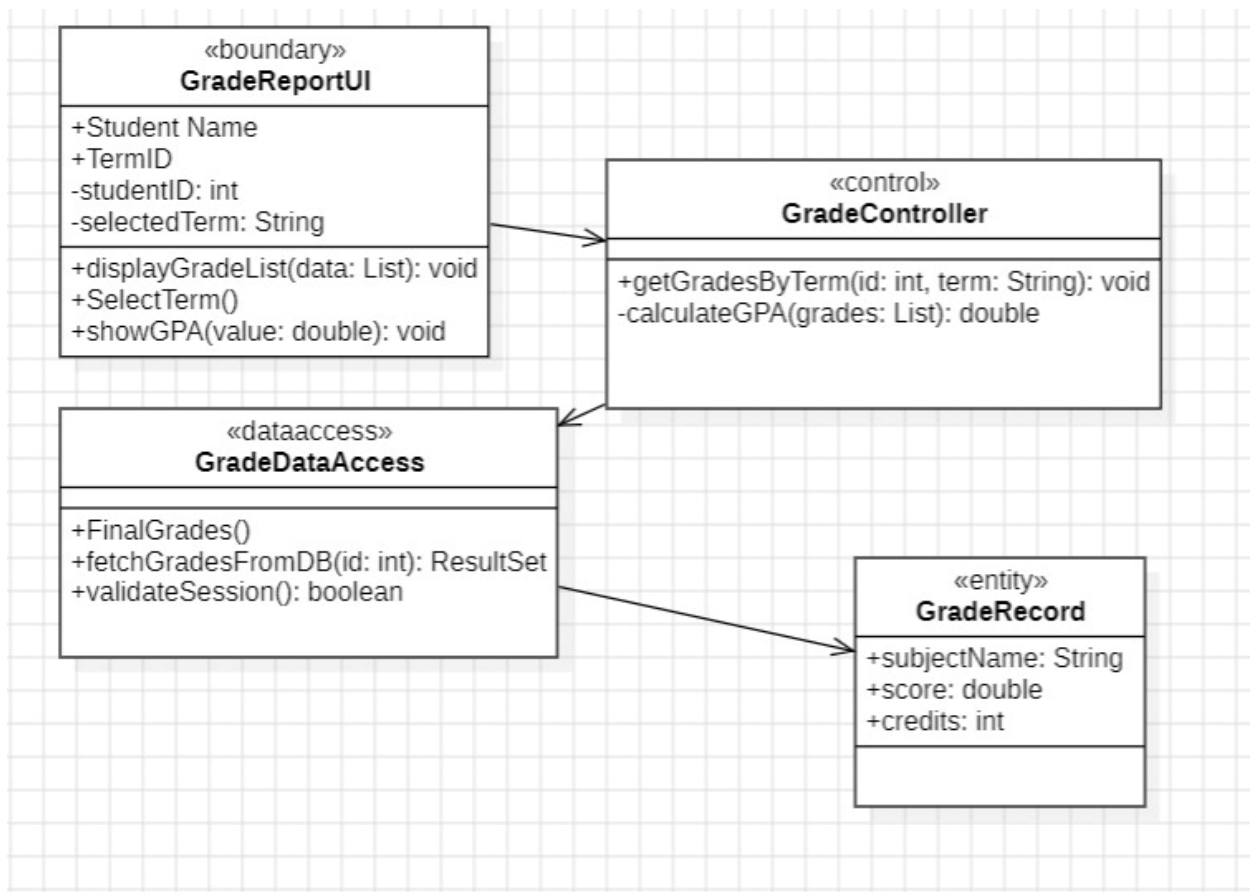
GradeReportUI	
Responsibilities	Collaborators
1. Provide interface for term selection and student ID input. 2. Format and render the grade table for display. 3. Display error messages if data retrieval fails.	GradeController

GradeController	
Responsibilities	Collaborators
• 1. Receive and validate term selection from the UI. • 2. Calculate the Term GPA based on retrieved scores. • 3. Manage the flow of grade data from data access to the boundary.	• GradeDataAccess • GradeReportUI

GradeDataAccess	
Responsibilities	Collaborators
• 1. Execute SQL queries to find grades filtered by Student ID. • 2. Manage database connection lifecycle (Open/Close). • 3. Transform raw database result sets into GradeRecord objects.	• GradeRecord

GradeRecord	
Responsibilities	Collaborators
1. Store persistent data including subject name, score, and credits. 2. Provide getter and setter methods for score manipulation. 3. Maintain data integrity for the specific grade instance.	GradeDataAccess

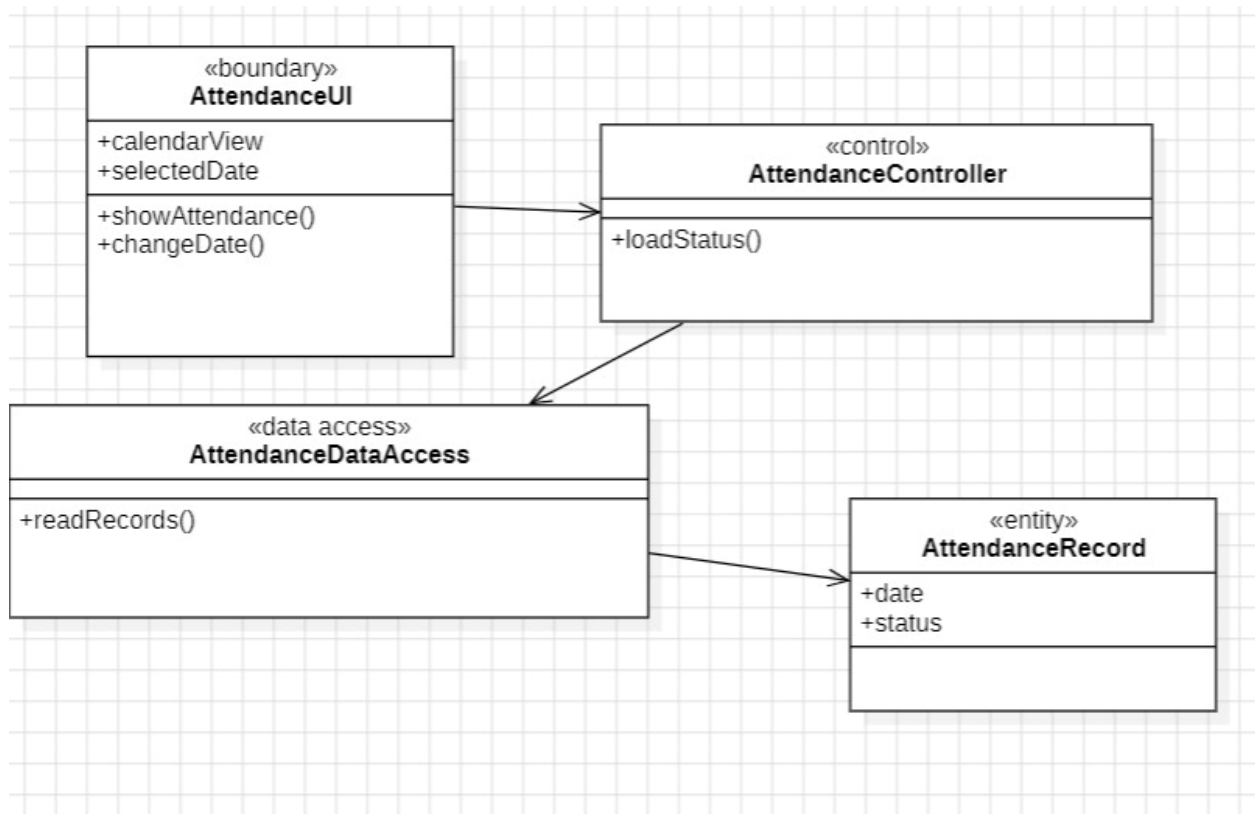
Updated Class Diagram :



2. Use Case Name: View Attendance (Simple)

- Scenario: User monitors daily attendance status (Present/Absent).
- Triggering event: User selects "Attendance History."
- Brief description: Displays a calendar or list view of a student's attendance records within a chosen date range.
- Actors: Primary: Student or Parent. Secondary: Teacher (who recorded it).
- Stakeholders: Students; Parents; School Administration.
- Related use cases: Record Attendance (attendance must be recorded first).
- Preconditions: * User is authorized to view the specific student's data.
 - AttendanceUI is initialized.
- Postconditions: * Attendance grid is rendered; total presence/absence summary is provided.
- Flow of activities:
 1. Actor selects a month or date range in the AttendanceUI.
 2. System (AttendanceController) requests records for that specific range.
 3. System (AttendanceDataAccess) retrieves AttendanceRecords from the DB.
 4. System computes the attendance percentage.
 5. System highlights absent days in red and present days in green on the UI.
- Exception conditions:
 - Invalid date range (e.g., end date before start date) → show error message.
 - Database connection failure → notify user to try again later.

First Cut Design Class Diagram :



CRC Cards :

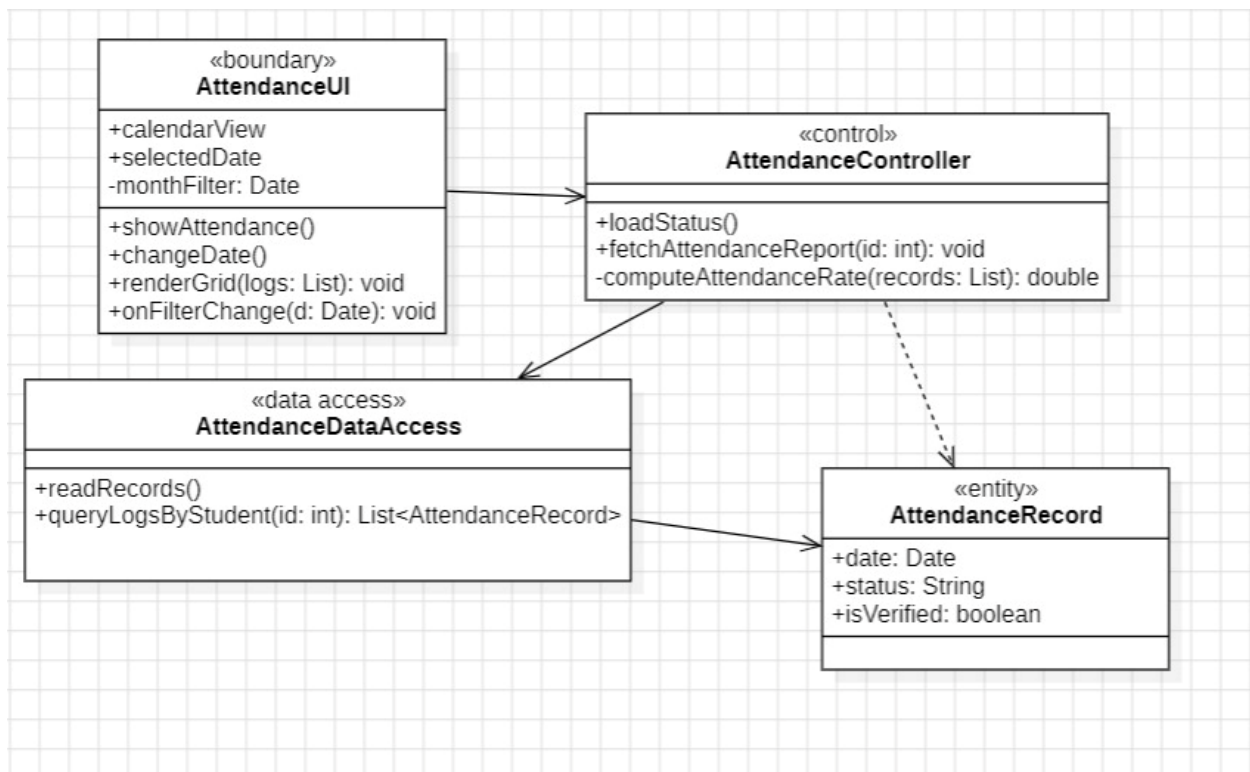
AttendanceUI	
Responsibilities	Collaborators
1. Display interactive calendar with attendance color-coding. 2. Capture date-range filters and month selections from the user. 3. Present a summary of total days present and absent.	AttendanceController

AttendanceController	
Responsibilities	Collaborators
• 1. Coordinate the search for attendance data based on date range. • 2. Calculate the overall attendance percentage for the term. • 3. Filter specific records (e.g., only "Excused Absences") if requested.	• AttendanceDataAccess • AttendanceUI

AttendanceDataAccess	
Responsibilities	Collaborators
• 1. Execute queries to fetch attendance logs for a specific student. • 2. Filter data at the database level using date range parameters. • 3. Handle data exceptions and empty result sets from the DB.	• AttendanceRecord

AttendanceRecord	
Responsibilities	Collaborators
1. Hold specific date, attendance status (P/A), and teacher remarks. 2. Categorize the type of record (e.g., Holiday vs School Day). 3. Provide timestamp information for when the record was last modified.	AttendanceDataAccess

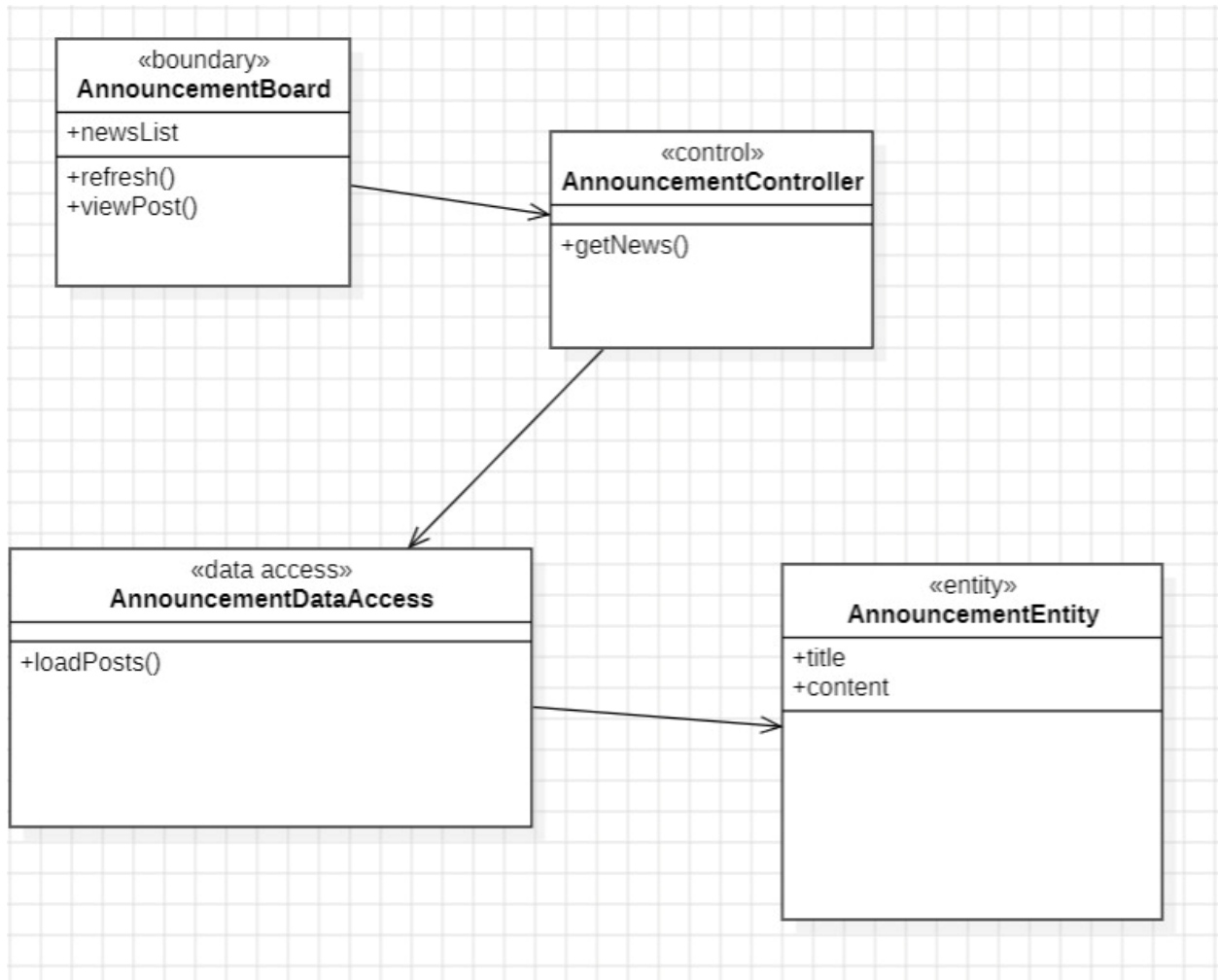
Updated Diagram :



3. Use Case Name: View Announcements (Simple)

- Scenario: User reads school news, updates, or urgent notices.
- Triggering event: User navigates to the “Announcements” section or clicks a notification.
- Brief description: Provides a filtered list of announcements relevant to the user's role (Student, Parent, or Teacher).
- Actors: Primary: All Users (Student, Parent, Teacher, Admin).
- Stakeholders: Entire School Community.
- Related use cases: Announcements and Communication (creation of the post).
- Preconditions: * User is logged in (to determine role-based filtering).
- Postconditions: * List of active announcements is displayed; unread posts are marked.
- Flow of activities:
 1. Actor requests to see the AnnouncementBoard.
 2. System (AnnouncementController) checks the user's role.
 3. System (AnnouncementDataAccess) fetches non-expired posts for that role.
 4. System sorts posts by urgency (priority) and date.
 5. System displays the titles; Actor clicks a title to view full content.
- Exception conditions:
 - No active announcements → show "No new announcements" message.

First Cut Design Class Diagram :



CRC Cards :

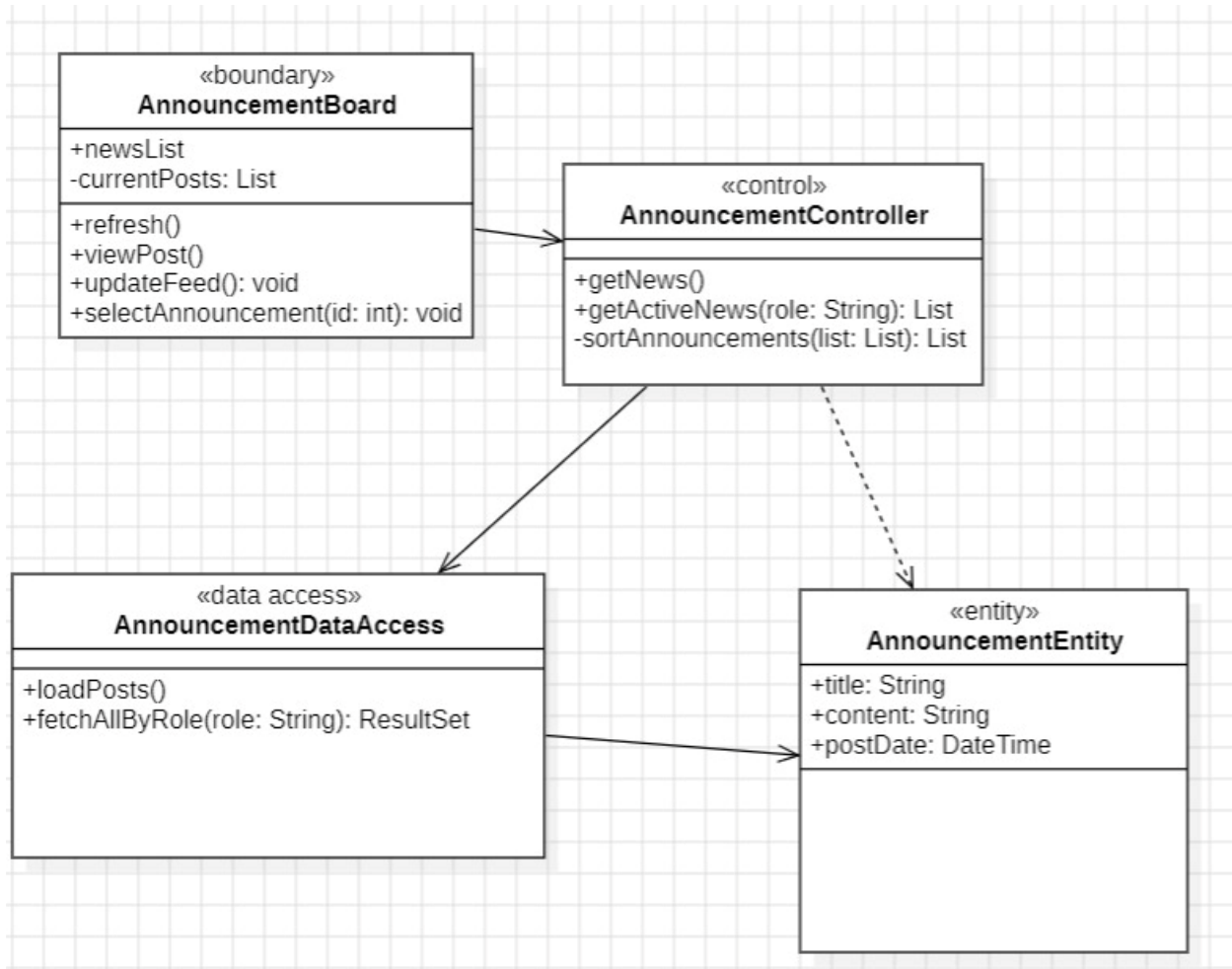
AnnouncementBoard	
Responsibilities	Collaborators
• 1. Render a scrollable list of school announcements and titles. • 2. Transition from the list view to a detailed individual post view. • 3. Manage UI states for "Unread" vs "Read" announcements.	• AnnouncementController

AnnouncementController	
Responsibilities	Collaborators
• 1. Filter news items according to the logged-in user's role. • 2. Sort posts by priority (Urgent) and chronological order. • 3. Remove/Hide announcements that have passed their expiry date.	• AnnouncementData Access • AnnouncementBoard

AnnouncementDataAccess	
Responsibilities	Collaborators
• 1. Retrieve all active post records from the persistent store. • 2. Perform keyword searches within the announcement database. • 3. Cache frequent announcements to improve system performance.	• AnnouncementEntity

AnnouncementEntity	
Responsibilities	Collaborators
• 1. Maintain the title, body text, and publication timestamp. • 2. Store metadata such as target audience (Parent, Student, All). • 3. Handle storage of any associated file paths for attachments.	• AnnouncementData Access

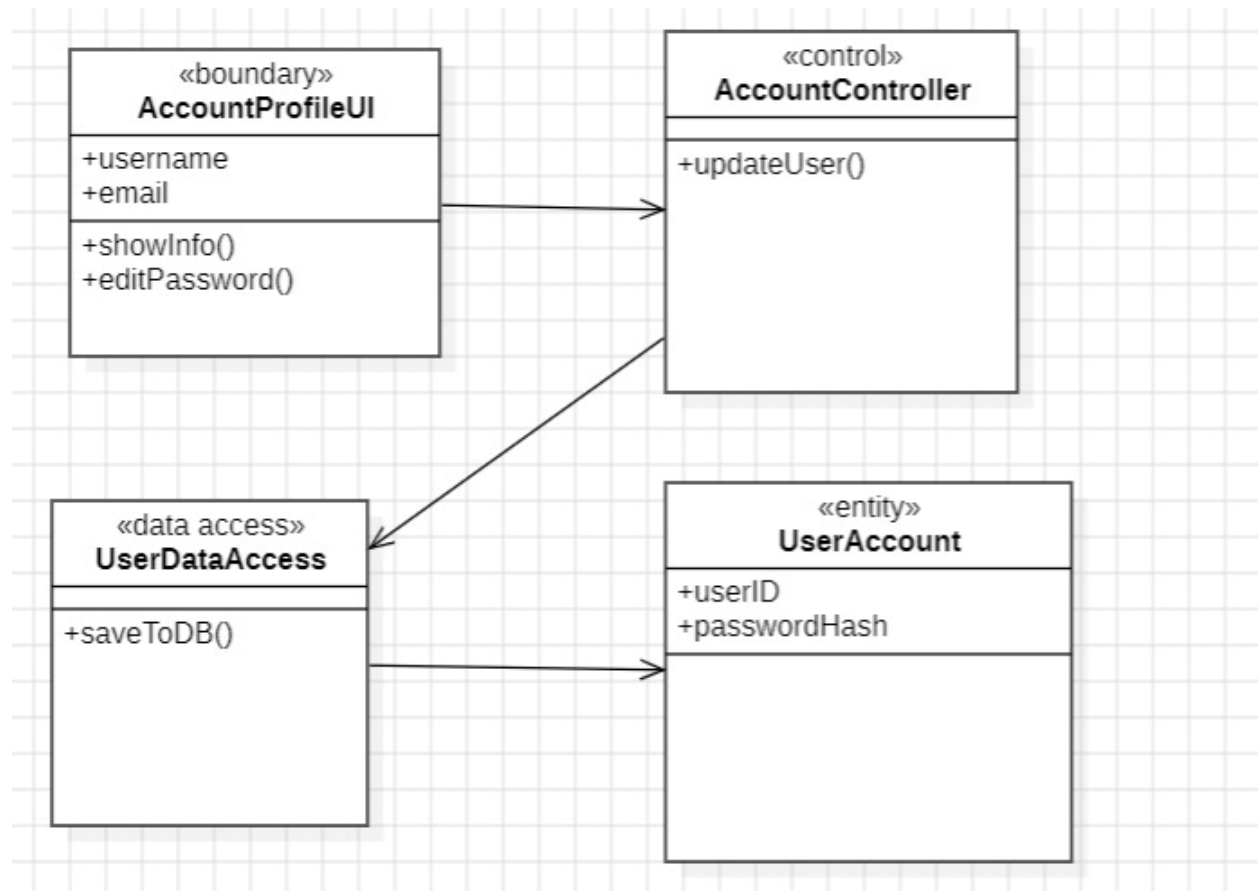
Updated Diagram :



4. Use Case Name: Manage User Accounts (Profile Update)(Simple)

- Scenario: User updates their personal profile information or password.
- Triggering event: User clicks on “Profile Settings.”
- Brief description: Allows users to view their account details and update their password or contact info securely.
- Actors: Primary: Any registered User (Admin, Teacher, Student, Parent).
- Stakeholders: System Admin; All Users; Security/Audit Team.
- Related use cases: Manage User Accounts (Admin-side management).
- Preconditions: * User is authenticated and currently active in the session.
- Postconditions: * UserAccount entity is updated in the database; password is re-hashed.
 - Security log of the change is recorded.
- Flow of activities:
 1. Actor requests “Profile Settings” and enters a new password or email.
 2. System validates password complexity and matching fields.
 3. System (AccountController) hashes the new password for security.
 4. System (UserDataAccess) commits the update to the UserAccount record.
 5. System confirms the success and sends a confirmation notification.
- Exception conditions:
 - Current password incorrect → reject change and notify actor.
 - New password does not meet complexity rules → show requirements error.

First Cut Design Class Diagram :



CRC Cards :

AccountProfileUI	
Responsibilities	Collaborators
1. Provide forms to display and edit user profile information. 2. Capture and validate password input for matching and length. 3. Display success or failure notifications after update attempts.	AccountController

AccountController	
Responsibilities	Collaborators
• 1. Implement security logic such as password hashing and encryption. • 2. Verify current user session and permissions before modifying. • 3. Trigger account log-out if critical security settings are changed.	• UserDataAccess • AccountProfileUI

UserDataAccess	
Responsibilities	Collaborators
• 1. Update existing user account records in the database safely. • 2. Check for duplicate usernames or email addresses in the DB. • 3. Generate and store logs of profile modification events.	• UserAccount

UserAccount	
Responsibilities	Collaborators
1. Store the primary userID, username, and encrypted hash. 2. Persist user-specific settings and application preferences. 3. Track account metadata like "Last Login" and "Account Status."	• UserDataAccess

Updated Diagram :

